

National Institute of Business Management (NIBM)
Coventry University (CU)

NB5004CEM - Cybersecurity

Methodology, SRS Writing & Shared Responsibility

Session 8 - The Pivot: From Understanding to Producing



Level 5 | 20 Credits

THE PIVOT SESSION

S8 - Methodology, SRS Writing & Shared Responsibility



All five layers are covered. Today, understanding becomes a written deliverable.

Session Overview

From building blocks to a written deliverable: your SRS starts today.

- Distinguish a security audit from a penetration test - and why the coursework is the former
- Write testable SRS requirements: ID, statement, acceptance test - all three, always
- Apply shared responsibility to decide what you may - and may not - write a requirement for
- Read a CVSS v3 score using the NVD calculator - no formulas required
- Leave with a working draft of your own SRS (A4) and target environment (B1)



Theory - 60 min

Method, not tools



Activity - 45 min

Write & peer review



Small Lab - 45 min

Your own SRS (direct deliverable)

Where This Session Fits: Module Learning Outcomes



LO1

Evaluate a range of encryption and authentication methods for a given set of requirements.

LO2

Evaluate different approaches to assessing a system's security and use these to evaluate the security of a system.

THIS SESSION

LO3

Utilise systematic knowledge to create secure environments at the host or network level.

LO4

Develop and evaluate software that addresses the most common and most severe security concerns.

Checkpoint: All Five Layers Are Now Covered

5

LAYERS NOW COVERED

IAM - Network - System - App - Data

12+

SRS IDs WRITABLE

Across all five layers

1

DELIVERABLE TODAY

Your own SRS skeleton (A4 + B1)

Layer	Session(s)	SRS Requirements Enabled
IAM	S4 + S5	IAM-01 (MFA), IAM-02 (RBAC)
Network	S7	NET-01, NET-02, NET-03
System	S7	SYS-01
Application	S6	APP-01, APP-02
Data	S3 + S7	DATA-01, DATA-02, DATA-03

KEY CONCEPT

This Is an Evaluation - Not a Penetration Test

A penetration test actively exploits a system to produce confirmed vulnerabilities. A security audit is a systematic review against requirements, standards and best practice - it needs no exploitation, and its scope is broader: policy, configuration and design, not just code. Your coursework is an evaluation.

Audit vs. Penetration Test



Penetration Test

- Goal: actively exploit and compromise
- Output: a list of confirmed vulnerabilities
- Scope: typically narrower and technical
- Requires: hands-on exploitation skill

Security Audit (This Coursework)

- Goal: systematically review against requirements
- Output: findings mapped to properties + SRS IDs
- Scope: broader - policy, config, design, code
- Requires: structured evidence and reasoning

Common misconception: you are not being asked to "hack" the system - you are being asked to evaluate it.

The Audit Lifecycle



- **Five stages: Scope -> Assets -> Evidence -> Risk Rating -> Reporting**
- **Scope defines exactly what is (and is not) being evaluated**
- **Evidence gathering draws on S9/S10 tools later - today is method, not tools**
- **Risk rating: severity per finding - CVSS supports this, coming up next**
- **The scope document is the single most important artefact - it protects the evaluator**

Analogy

A scope document is like a building surveyor's brief: it fixes exactly which rooms will be inspected before the surveyor sets foot inside - so no one can later argue about what was, or wasn't, checked.

KEY CONCEPT

A Requirement Isn't Testable Unless It Has All Three

ID + Statement + Acceptance Test. If another evaluator could not verify it, it is not a requirement - it's a hope. Testability is the whole point: it is what lets every Part B control, and your viva defence, trace back to something concrete.

Anatomy of a Testable SRS Requirement



- **ID** - a short reference code, e.g. IAM-01, NET-02
- **Statement** - specifies exactly what must be true
- **Acceptance test** - exactly how compliance is verified
- **All three parts are non-negotiable** - miss one and it isn't testable

Analogy

Think of it like a recipe with a taste test built in: the ID is the recipe's name, the statement is the ingredient list and method, and the acceptance test is "it's done when..." - without that last part, no one else can confirm you got it right.

Three Worked Examples



IAM-01

IAM LAYER

Statement: All administrative accounts must require MFA.

Test: Attempt to authenticate to the admin panel with valid credentials but without the second factor - authentication must be denied.

NET-02

NETWORK LAYER

Statement: A WAF must be deployed in front of all public-facing application endpoints.

Test: Send a request containing a known SQLi pattern - the WAF must block it and log the attempt.

DATA-01

DATA LAYER

Statement: All personally identifiable data at rest must be encrypted using AES-256 or equivalent.

Test: Access the database storage volume directly without application credentials - data must be unreadable.

When a Requirement Fails the Test



- **VAGUE:** "The system should have strong password policies."
- **NOT TESTABLE** - strong by what measure? Checked how?
- **TESTABLE:** IAM-03 - passwords hashed with bcrypt (cost ≥ 12). Test: inspect the stored hash format and cost factor.
- **VAGUE:** "The application should be secure against injection."
- **NOT TESTABLE** - secure against what, verified how?
- **TESTABLE:** APP-03 - all user input parameterised before reaching the database. Test: submit a known SQLi payload - no query change, no data leak.

Shared Responsibility, In Depth



- **IaaS** - provider secures infrastructure and hypervisor; you secure OS, app, data
- **PaaS** - provider secures runtime and infrastructure; you secure app and data
- **SaaS** - provider secures most of the stack; you secure data, access, configuration
- **Misunderstanding this boundary is a leading cause of cloud security failures**

Analogy

Think of it like renting an apartment: the landlord (provider) maintains the building's structure and utilities, but you (the customer) are responsible for locking your own door and how you use the space inside.

KEY CONCEPT



You May Only Write Requirements for What You Own

If the cloud provider owns a layer of the stack, writing an SRS requirement against it is not a weak requirement - it is a scope error. Know your deployment model (IaaS / PaaS / SaaS) before you write a single requirement.

CVSS v3: Read the Score, Don't Derive It



- CVSS v3 gives a finding a severity score and band (Critical / High / Medium / Low)
- Use the NVD online calculator only - select the correct value for each metric
- No formula memorisation, no manual maths - it's selection-based, not mathematical
- A provisional score supports the risk-rating stage of the audit lifecycle

Analogy

Like a triage nurse working from a checklist rather than a diagnosis from first principles: you select the observed factors, and the tool returns the priority.

Your Domain, Your System



- Recall your assigned domain: student index mod 6
- 0 Banking - 1 Healthcare - 2 Transport/Aviation - 3 Education - 4 Ecommerce/Retail - 5 Hospitality
- The system description ahead reflects YOUR assigned domain
- Everything from here uses your system - not a generic example
- What you produce next heads straight into your coursework

Activity: Write & Peer-Review 3 SRS Requirements

- ✦ Write 3 testable SRS requirements distributed across at least 3 different layers
- ✦ Each requirement must have an ID and an acceptance test
- ✦ Peer review: would this requirement actually be testable by another evaluator?



Does the Provider Own This - Or You?

Which of your requirements are weakened - or invalidated - by shared responsibility? For each one, ask: does the cloud provider own this, or do you? Be ready to defend your answer to the group.

Small Lab: Your Coursework SRS Skeleton

- ✦ **System context - 2-3 sentences: purpose, critical assets, primary roles**
- ✦ **State your target environment and deployment model**
- ✦ **At least 6 SRS requirements across the five layers, each with an ID and acceptance criterion**
- ✦ **This is a working draft of A4 (mini-SRS) and the start of B1 - not a rehearsal**



Try It: SRS Testability Validator + CVSS Explorer



- **Panel A** - type a draft requirement and see exactly what it's missing
- **Panel B** - drag the CVSS metric values and watch the score and severity band move
- Try writing a deliberately vague requirement first - watch it fail - then fix it
- Try scoring a finding from your own domain's CVE research (S1) in the CVSS panel

Coursework Hook: What You Leave With Today



Part A - A4 Mini-SRS (10%)

- System context: purpose, assets, roles
- Target environment and deployment model
- 6+ testable SRS requirements, five layers
- Each with an ID and acceptance criterion

Part B - B1 Target Environment

- Target environment description drafted
- Shared responsibility boundary stated
- Foundation for every B2-B8 control
- Every later control traces back here

Session Recap: Five Anchors Check



- **Properties** - testability is how Accountability becomes verifiable, across all five
- **Layers** - IAM, Network, System, Application, Data are all now writable as SRS requirements
- **STRIDE** -> property - not revisited today; assumed background from S2
- **Domain assignment** - honoured directly: your lab used your own domain's system
- **Coursework hook** - A4 (mini-SRS) and B1 (target environment) drafted today
- **Next**: S9-S10 gather evidence, S11 fills the gaps, S12 turns it into your report

Questions & Discussion



Before you go: is your SRS skeleton saved somewhere you can find it next session?



Disclaimer & Copyrights



The information presented in this presentation is intended for general informational purposes only. It is not meant to be comprehensive or to constitute professional advice. Any reliance you place on such information is strictly at your own risk. While I strive to keep the information accurate and up-to-date, I make no representations or warranties of any kind, express or implied, about the completeness, accuracy, reliability, suitability, or availability concerning the content within this presentation.

This presentation may include references or links to third-party websites or content. I do not endorse the views expressed or the information provided on such external sites. I am not responsible for the content of any linked site or any link contained in a linked site.

Any action you take upon the information presented in this presentation is strictly at your own discretion. I will not be liable for any losses or damages in connection with the use of this information.

This disclaimer is subject to change without notice.

@ Images and artworks used in this presentation are used for general informational purposes only. It may contain copyrighted material, the use of which may not have been specifically authorised by the copyright owner. This material is available in an effort to explain issues relevant to the topic. This should constitute a 'fair use' of any such copyrighted material.

Last modified: 4 Jul 2026